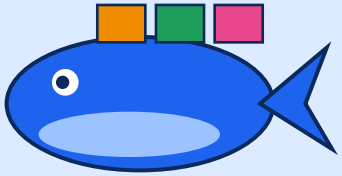
Riya's Packing Studio

Writing the Recipe Card (Dockerfile)

Docker Episode 3 as a story. No tech knowledge needed!

Week 2 - Episode 3 - Story Edition



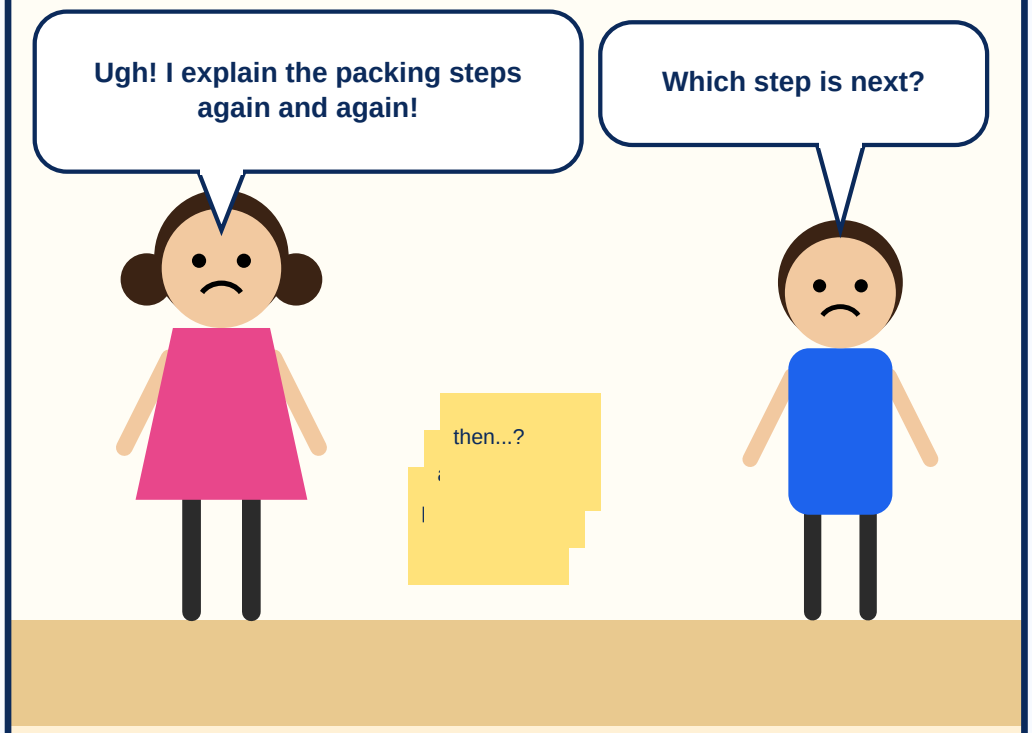
Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
CMD python app.py
```

Riya writes her packing steps once on a Recipe Card, and a machine packs the box every time.

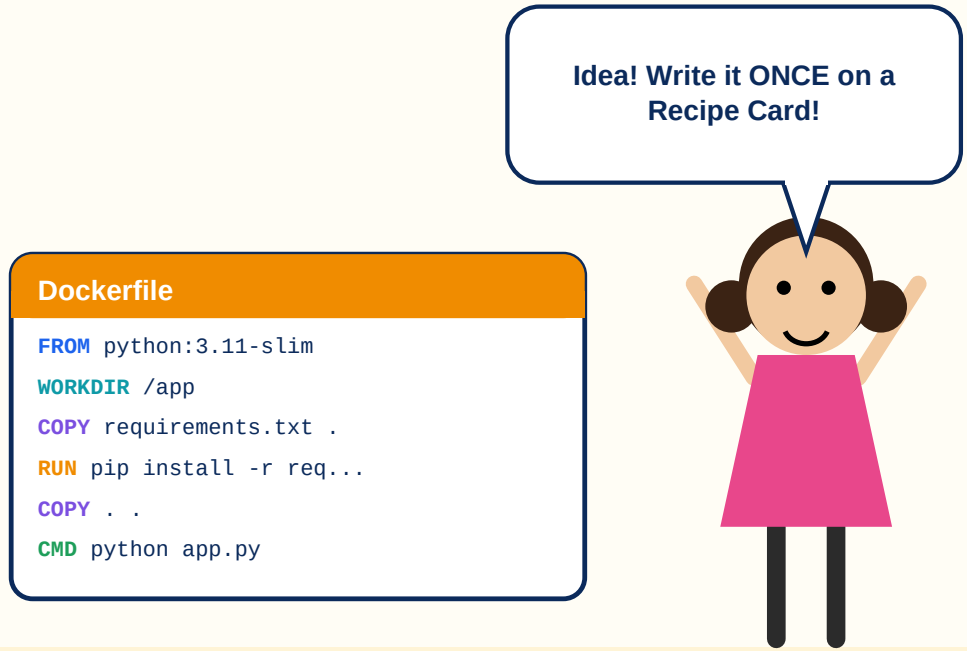
Part 1: The Problem and the Idea

1 Packing by hand



Riya packs her cake kits by hand and keeps repeating the same steps. TECH: building environments manually is slow and error-prone.

2 The Recipe Card = Dockerfile



The fix: write the packing steps ONCE on a Recipe Card. TECH: a Dockerfile is a text file of instructions that builds an image.

Part 2: Starting Point and Workbench

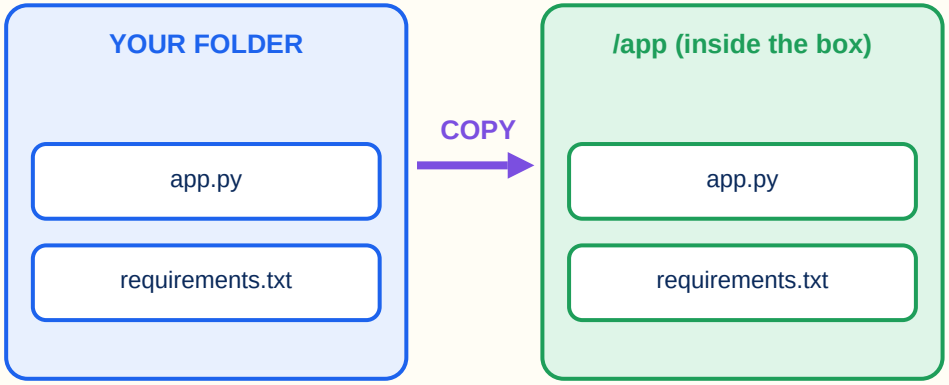
3

```
FROM python:3.11-slim
```

FROM picks a ready-made base dough. TECH: FROM python:3.11-slim starts your image from an official Python base so you do not build from scratch.

4

```
WORKDIR /app
COPY . .
```



WORKDIR = the workbench folder inside the box

WORKDIR sets the workbench folder inside the box. COPY brings your ingredients (app files) from your folder into it.

Part 3: Factory Time vs Table Time

5

AT THE FACTORY
(build time, ONCE)

```
RUN pip install
  -r requirements.txt
```

RUN prepares the ingredients while packing. They are baked into the box.

AT THE TABLE
(run time, EVERY open)

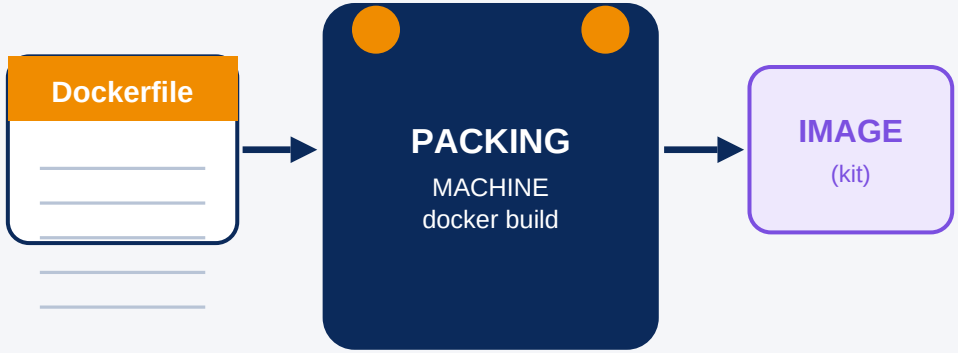
```
CMD ["python",
  "app.py"]
```

CMD says what to do when a customer opens the box: start the app.

RUN happens at the factory while packing (once). CMD happens at the table when the box is opened (every time a container starts).

6

```
docker build -t hello-flask:1.0 .
```



-t = name:tag . = the folder to pack from

docker build is the packing machine: the Recipe Card goes in, a sealed kit (image) comes out. -t gives it a name and tag, and . is the folder to use.

Part 4: Layers and Choices

7

1st build	2nd build (code changed)
COPY code NEW	**COPY code REBUILT**
RUN pip NEW	RUN pip CACHED
COPY req NEW	COPY req CACHED
WORKDIR NEW	WORKDIR CACHED
FROM NEW	FROM CACHED

Only the changed layer and those after it are redone: fast!

Each step becomes a layer, like a layer cake. Docker remembers finished layers (cache) and redoes only what changed, so rebuilds are quick.

8

CMD = a suggestion

```
CMD python app.py
```

Customer types:

```
docker run IMG
python --version
```

↓

The suggestion is REPLACED.
Runs: python --version

ENTRYPOINT = fixed

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Customer types:

```
docker run IMG
other.py
```

↓

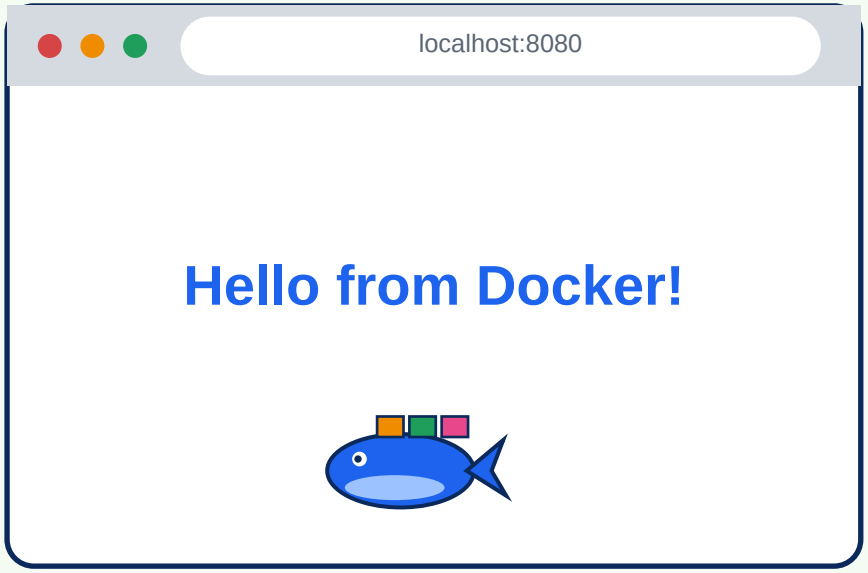
Extra words are ADDED.
Runs: python other.py

CMD is a default the customer can replace. ENTRYPOINT is the fixed machine and extra words are added to it. Often they are used together.

Part 5: Serving the Result

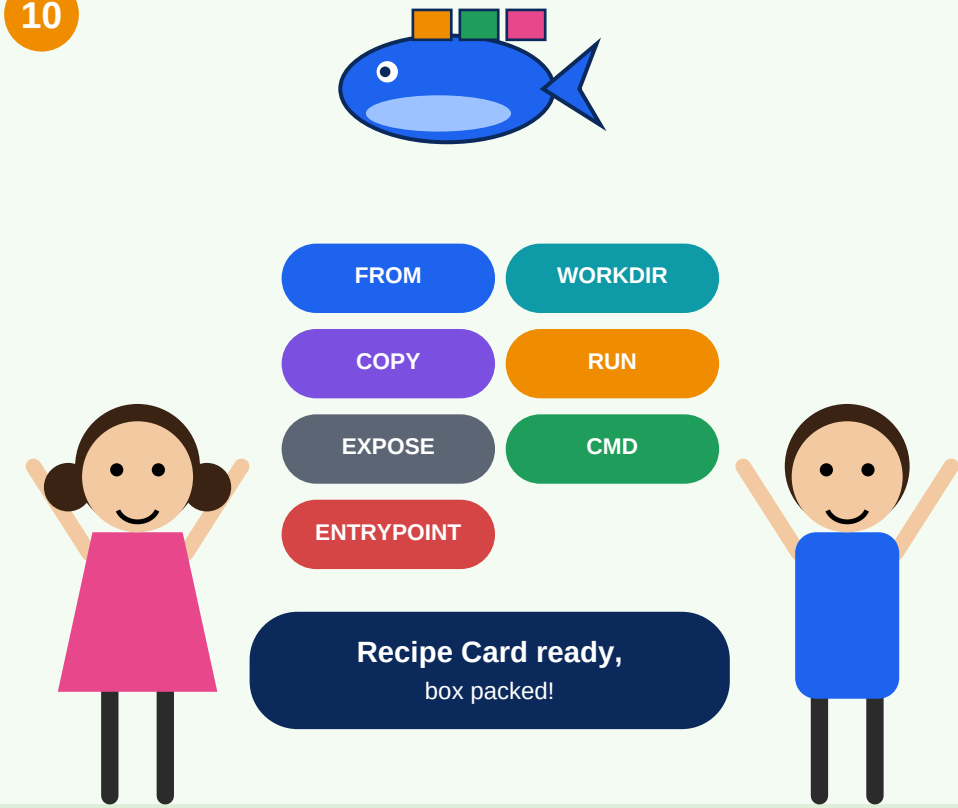
9

```
docker run -d -p 8080:5000 hello-flask:1.0
```



Run the kit and open the browser: your own app answers. -p 8080:5000 connects your window 8080 to the app's port 5000 inside.

10



Recipe Card: FROM, WORKDIR, COPY, RUN, EXPOSE, CMD, ENTRYPOINT. Now you can pack your own app into an image anyone can run.

Story-to-Tech Dictionary

In the story	In real tech
Recipe Card	Dockerfile
Ready-made base dough	FROM (base image)
The workbench	WORKDIR
Bringing ingredients to the bench	COPY
Prep at the factory	RUN (build time)
What happens when the box opens	CMD (run time)
Fixed machine in the box	ENTRYPOINT
Sign: "we serve at window 5000"	EXPOSE (documentation only)
Packing machine	docker build
Name and version sticker	-t name:tag
Sealed recipe kit	Image
Layer cake	Image layers
Remembering finished layers	Build cache
Opened box	Container

Remember

- A Dockerfile is the recipe for an image.
- docker build makes the image; docker run makes a container.
- RUN = build time. CMD = run time.
- Put things that change often LAST for fast rebuilds.
- EXPOSE only documents a port. -p publishes it.
- Edit the Dockerfile? Rebuild the image.